

Artificial Intelligence

Dr. Suman Kumar Maji

Indian Institute of Technology Patna



Informed Search

Informed Search

Heuristic Search
Heuristic Function
 $h(n)$
Types of Heuristic
Function $h(n)$
Admissible Heuristic
Function $h(n)$
Non-Admissible
Heuristic Function
 $h(n)$

Best First Search

A* Search

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

Definition

Informed search uses additional knowledge about the problem to guide the search toward the goal efficiently. This knowledge is provided using a **heuristic function**.

Evaluation Function

$$f(n) = g(n) + h(n)$$

Terminologies

- ◇ $g(n)$ – **Path Cost:** Actual cost from the start node to node n .
- ◇ $h(n)$ – **Heuristic Cost:** Estimated cost from node n to the goal.
- ◇ $f(n)$ – **Evaluation Function:** Estimated total cost of a solution path through node n .

Properties

- ◇ Uses information about the **goal state**.
- ◇ Explores fewer nodes compared to uninformed search.
- ◇ Guides the search toward promising nodes.
- ◇ Examples: **Best First Search, A* Search.**



Heuristic Search

Informed Search

Heuristic Search

Heuristic Function
 $h(n)$

Types of Heuristic
Function $h(n)$

Admissible Heuristic
Function $h(n)$

Non-Admissible
Heuristic Function
 $h(n)$

Best First Search

A* Search

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

- ◇ Heuristic Search and Heuristic Function are used in informed search.
- ◇ Heuristic search is a simple searching technique that tries to **optimize** a problem using a **Heuristic Function**.
- ◇ **Optimization** means that we try to solve a problem in minimum number of steps or minimum cost.



Heuristic Function $h(n)$

Informed Search

Heuristic Search

Heuristic Function $h(n)$

Types of Heuristic Function $h(n)$

Admissible Heuristic Function $h(n)$

Non-Admissible Heuristic Function $h(n)$

Best First Search

A* Search

Hill Climbing Search

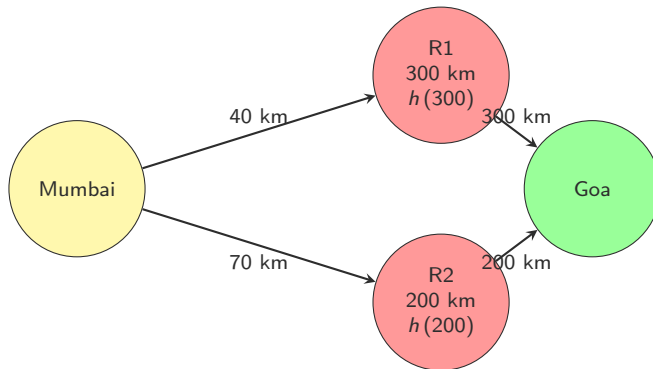
Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

- ◇ It is a function $h(n)$ that estimates the cost of reaching the goal from node n .
- ◇ It helps in selecting the optimal node for expansion.



Actual distance:

$R1 = 340 \text{ km}$

$R2 = 270 \text{ km}$



Types of Heuristic Function $h(n)$

Informed Search

Heuristic Search

Heuristic Function

$h(n)$

Types of Heuristic Function $h(n)$

Admissible Heuristic Function $h(n)$

Non-Admissible Heuristic Function $h(n)$

Best First Search

A* Search

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

Types of Heuristic Functions

There are two types of heuristic functions:

- ◇ Admissible
- ◇ Non-Admissible

Admissible Heuristic Function

- ◇ A heuristic function is admissible if it never overestimates the cost of reaching the goal.

$$h(n) \leq h^*(n)$$

- ◇ $h(n)$: heuristic cost
- ◇ $h^*(n)$: true cost to goal
- ◇ Heuristic cost should be less than or equal to the true cost.

Non-Admissible Heuristic Function

- ◇ A non-admissible heuristic may overestimate the cost of reaching the goal.

$$h(n) > h^*(n)$$

- ◇ $h(n)$: heuristic cost
- ◇ $h^*(n)$: true cost
- ◇ Heuristic may be greater than the actual cost.

Admissible Heuristic Function $h(n)$



Informed Search

Heuristic Search

Heuristic Function

$h(n)$

Types of Heuristic

Function $h(n)$

Admissible Heuristic

Function $h(n)$

Non-Admissible

Heuristic Function

$h(n)$

Best First Search

A* Search

Hill Climbing

Search

Game Tree

Search: The

Mini-Max

Algorithm

Alpha-Beta

Pruning

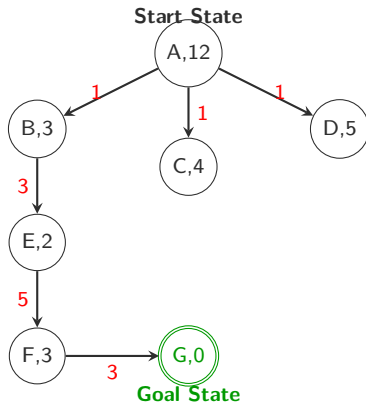
Constraint

Satisfaction

Problem

Cryptarithmic

Problems



Node format : (node , heuristic)

Heuristic Values

- ◇ $h(B)=3$
- ◇ $h(C)=4$
- ◇ $h(D)=5$
- ◇ $h(E)=2$
- ◇ $h(F)=3$

Evaluation Function

$$f(n) = g(n) + h(n)$$

Cost = Actual cost + Heuristic cost

Actual path cost from A to G

$$1 + 3 + 5 + 3 = 12$$

Admissible Heuristic



Admissible condition

$$h(n) \leq h^*(n)$$

True Cost to Goal

- ◇ $h^*(F) = 3$
- ◇ $h^*(E) = 5 + 3 = 8$
- ◇ $h^*(B) = 3 + 5 + 3 = 11$
- ◇ $h^*(C) = \infty$ (no path to goal)
- ◇ $h^*(D) = \infty$ (no path to goal)

Check $h(n) \leq h^*(n)$

- ◇ B : $3 \leq 11$
- ◇ E : $2 \leq 8$
- ◇ F : $3 \leq 3$
- ◇ C : $4 \leq \infty$
- ◇ D : $5 \leq \infty$

Result

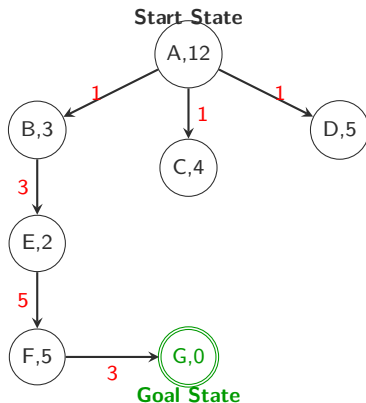
All nodes satisfy admissibility

⇒ **Heuristic is Admissible**

Non-Admissible Heuristic Function $h(n)$



Informed Search
Heuristic Search
Heuristic Function $h(n)$
Types of Heuristic Function $h(n)$
Admissible Heuristic Function $h(n)$
Non-Admissible Heuristic Function $h(n)$
Best First Search
A* Search
Hill Climbing Search
Game Tree Search: The Mini-Max Algorithm
Alpha-Beta Pruning
Constraint Satisfaction Problem
Cryptarithmic Problems



Node format : (node , heuristic)

Heuristic Values

- ◇ $h(B)=3$
- ◇ $h(C)=4$
- ◇ $h(D)=5$
- ◇ $h(E)=2$
- ◇ $h(F)=5$

Evaluation Function

$$f(n) = g(n) + h(n)$$

Cost = Actual cost + Heuristic cost

Actual path cost from A to G

$$1 + 3 + 5 + 3 = 12$$

Non-Admissible Heuristic



Informed Search

Heuristic Search

Heuristic Function

$h(n)$

Types of Heuristic

Function $h(n)$

Admissible Heuristic

Function $h(n)$

Non-Admissible

Heuristic Function

$h(n)$

Best First Search

A* Search

Hill Climbing

Search

Game Tree

Search: The

Mini-Max

Algorithm

Alpha-Beta

Pruning

Constraint

Satisfaction

Problem

Cryptarithmic

Problems

Non-admissible condition

$$h(n) > h^*(n)$$

True Cost to Goal

- ◇ $h^*(F) = 3$
- ◇ $h^*(E) = 5 + 3 = 8$
- ◇ $h^*(B) = 3 + 5 + 3 = 11$
- ◇ $h^*(C) = \infty$
- ◇ $h^*(D) = \infty$

Check $h(n) \leq h^*(n)$

- ◇ B : $3 \leq 11$
- ◇ E : $2 \leq 8$
- ◇ F : $5 > 3$
- ◇ C : $4 \leq \infty$
- ◇ D : $5 \leq \infty$

Result

Node F violates admissibility

$\Rightarrow$ **Heuristic is Non-Admissible**

Best First Search



Best First Search

Informed Search

Best First Search

Best First Search
Example

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Definition

Best First Search expands the node that appears most promising using a heuristic evaluation function.

Key Idea

- ◇ Uses an evaluation function to choose the next node.
- ◇ Always selects the node closest to the goal.
- ◇ Belongs to the category of **Informed (Heuristic) Search**.

Evaluation Function

$$f(n) = h(n)$$

- ◇ $h(n)$: heuristic estimate from node n to the goal.

Properties

- ◇ Expands the most promising node first.
- ◇ Uses heuristic information to guide the search.
- ◇ Faster than uninformed search but not always optimal.



Best First Search Example

Informed Search

Best First Search

Best First Search
Example

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

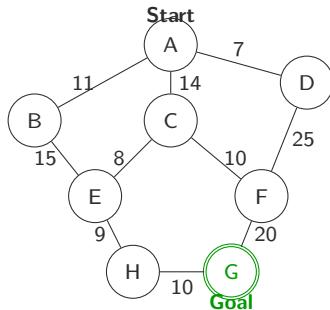
Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Question

Apply **Best First Search** to find a path from the **Start node A** to the **Goal node G**. Use the **Straight Line Distance (SLD)** to the goal as the heuristic function $h(n)$.



Straight Line Distance

$$h(n) = \text{SLD from node } n \text{ to Goal } G$$

Node	$h(n)$
A	40
B	32
C	25
D	35
E	19
F	17
H	10
G	0



Best First Search Example

Informed Search

Best First Search

Best First Search
Example

A* Search

Hill Climbing
Search

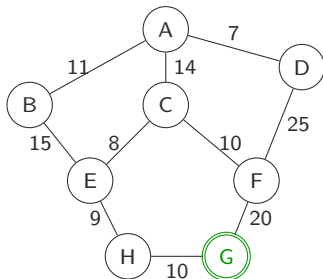
Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Graph and Heuristic Table



SLD Heuristic

Node	$h(n)$
A	40
B	32
C	25
D	35
E	19
F	17
H	10
G	0

(node, $h(n)$), O-OPEN, C-CLOSED

St. Outcome

1. C={}, O={(A,40)}

2. C={A}, O={(B,32),(C,25),(D,35)}

3. C={A,C},
O={(B,32),(D,35),(F,17),(E,19)}

4. C={A,C,F},
O={(B,32),(D,35),(E,19),(G,0)}

5. C={A,C,F,G}, O={(B,32),(D,35),(E,19)}

Goal G found

Path: A → C → F → G



Best First Search Example

Informed Search

Best First Search

Best First Search
Example

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

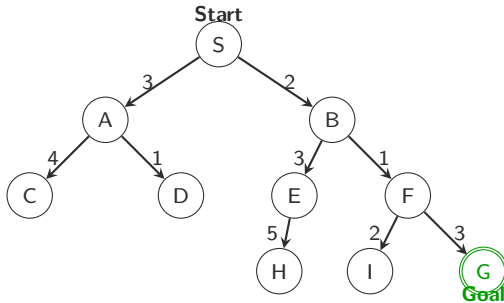
Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Question

Apply **Best First Search** to find a path from the **Start node S** to the **Goal node G**. Use the following heuristic values $h(n)$.



Heuristic Values

Node	$h(n)$
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0



Best First Search Solution

Informed Search

Best First Search

Best First Search
Example

A* Search

Hill Climbing
Search

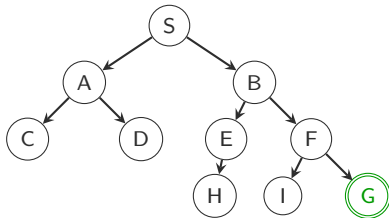
Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Graph



Heuristic Table

Node	$h(n)$
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

(node, $h(n)$), O-OPEN, C-CLOSED

St. Outcome

1. C={}, O={(S,13)}

2. C={S}, O={(A,12),(B,4)}

3. C={S,B}, O={(A,12),(F,2),(E,8)}

4. C={S,B,F}, O={(A,12),(E,8),(G,0),(I,9)}

5. C={S,B,F,G} O={(A,12),(E,8),(I,9)}

Goal G found

Path: S → B → F → G



Evaluation Function

$$f(n) = h(n)$$

Best First Search selects the node with the **minimum heuristic value**.

Time Complexity

- ◇ Worst case: $O(b^m)$
- ◇ b = branching factor
- ◇ m = maximum depth of the search tree

Completeness

- ◇ Not always complete
- ◇ May fail in infinite graphs
- ◇ Depends on heuristic function

Space Complexity

- ◇ $O(b^m)$
- ◇ Stores all generated nodes in memory (OPEN list)

Optimality

- ◇ Not optimal
- ◇ Uses only heuristic $h(n)$
- ◇ Ignores actual path cost $g(n)$

A* Search

A* Search – Practice Graph with Heuristic Values



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

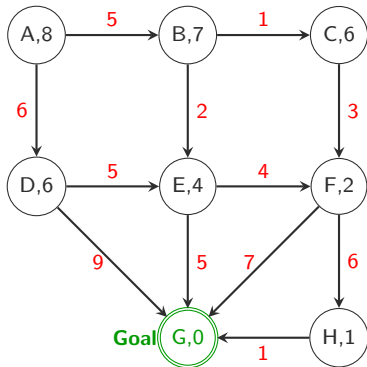
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Start Node: A

Notation

- ◇ $g(n)$: cost from start node
- ◇ $h(n)$: heuristic estimate to goal
- ◇ $f(n) = g(n) + h(n)$

Each node is written as:

[(node, $g(n)$, $h(n)$, $f(n)$)]

Example:

[(A, 0, 8, 8)]

means heuristic estimate from A to goal is 8.

Goal node always has

[$h(G)=0$]



A* Search

Informed Search

Best First Search

A* Search

8-Puzzle Problem –
Introduction

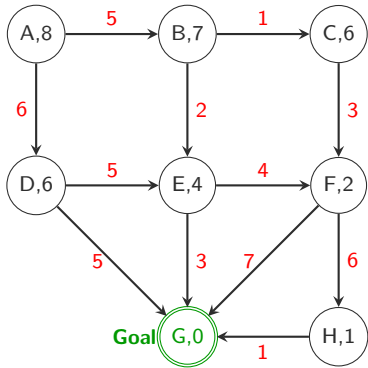
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Start Node: A

(node,g(n),h(n),f(n)), O-OPEN, C-CLOSED

St. Outcome

1. C={}, O={(A,0,8,8)}



A* Search

Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

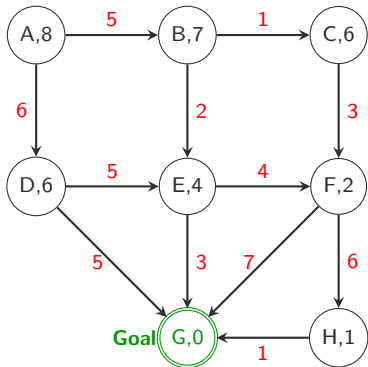
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Start Node: A

(node,g(n),h(n),f(n)), O–OPEN, C–CLOSED

St. Outcome

1. C={}, O={(A,0,8,8)}

2. C={A}, O={(B,5,7,12),(D,6,6,12)}



A* Search

Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

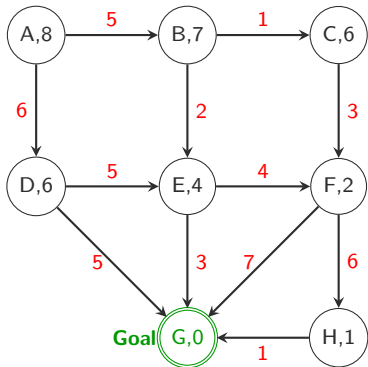
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



(node,g(n),h(n),f(n)), O-OPEN, C-CLOSED

St. Outcome

1. C={}, O={(A,0,8,8)}

2. C={A}, O={(B,5,7,12),(D,6,6,12)}

3. C={A,B}, O={(D,6,6,12),(E,7,4,11),(C,6,6,12)}



A* Search

Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

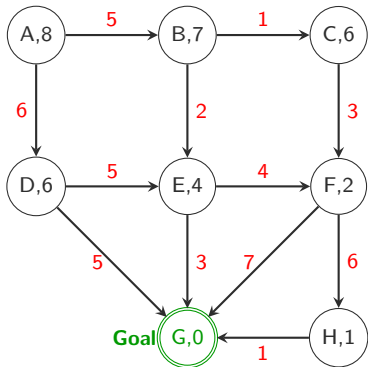
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



(node,g(n),h(n),f(n)), O-OPEN, C-CLOSED

St. Outcome

1. C={}, O={(A,0,8,8)}

2. C={A}, O={(B,5,7,12),(D,6,6,12)}

3. C={A,B}, O={(D,6,6,12),(E,7,4,11),(C,6,6,12)}

4. C={A,B,E},
O={(C,6,6,12),(D,6,6,12),(G,10,0,10),(F,11,2,13)}



A* Search

Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

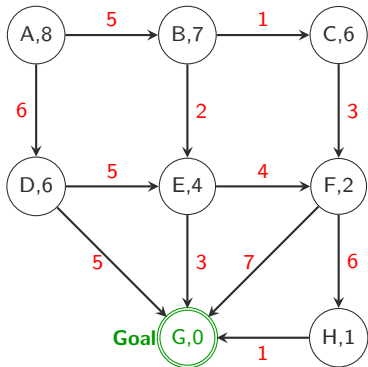
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



(node,g(n),h(n),f(n)), O-OPEN, C-CLOSED

St. Outcome

1. C={}, O={(A,0,8,8)}

2. C={A}, O={(B,5,7,12),(D,6,6,12)}

3. C={A,B}, O={(E,7,4,11),(C,6,6,12),(D,6,6,12)}

4. C={A,B,E},
O={(C,6,6,12),(D,6,6,12),(G,10,0,10),(F,11,2,13)}

5. C={A,B,E,G},
O={(C,6,6,12),(D,6,6,12),(F,11,2,13)} **Goal G found**

Terminate with cost = 10

A* Search - Optimal Path



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

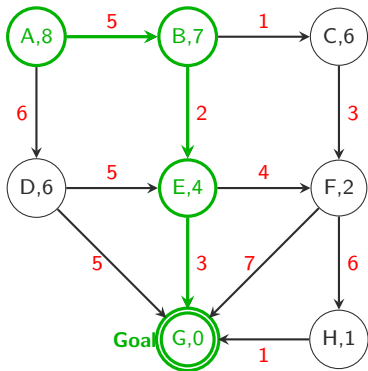
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Start Node: A

(node,g(n),h(n),f(n)), O–OPEN, C–CLOSED

St. Outcome

1. C={}, O={(A,0,8,8)}

2. C={A}, O={(B,5,7,12),(D,6,6,12)}

3. C={A,B},
O={(E,7,4,11),(C,6,6,12),(D,6,6,12)}

4. C={A,B,E},
O={(C,6,6,12),(D,6,6,12), (G,10,0,10),(F,11,2,13)}

5. C={A,B,E,G}

Goal G found

Terminate with cost = 10

Backtracking: $G \leftarrow E \leftarrow B \leftarrow A$

Optimal Path: $A \rightarrow B \rightarrow E \rightarrow G$



Optimal Path in A* Search

Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

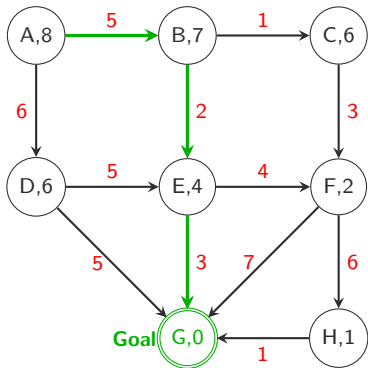
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Start Node: A

Goal Node Selected

The goal node G is selected from the OPEN list with:

$$g(G) = 10$$

Tracing the Optimal Path

Each node keeps a pointer to the node from which it was generated and updated the cost .

- ◇ G generated from E
- ◇ E generated from B
- ◇ B generated from A

Tracing backwards:

$$G \leftarrow E \leftarrow B \leftarrow A$$

Optimal Path

$$A \rightarrow B \rightarrow E \rightarrow G$$

A* Search – Example 1



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

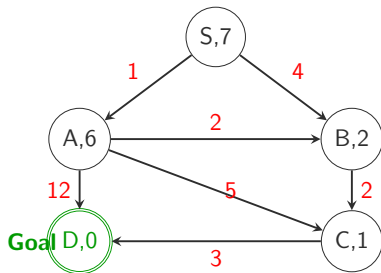
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Start Node: S

A* Search Problem

Apply the **A* Search Algorithm** to find the optimal path cost from **Start node S** to the **Goal node D**.

Evaluation Function

$$f(n) = g(n) + h(n)$$

- ◇ $g(n)$: cost from start node
- ◇ $h(n)$: heuristic estimate to goal
- ◇ $f(n)$: evaluation value

Each node is written as:

$$(node, g(n), h(n), f(n))$$

A* Search-Example 1



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

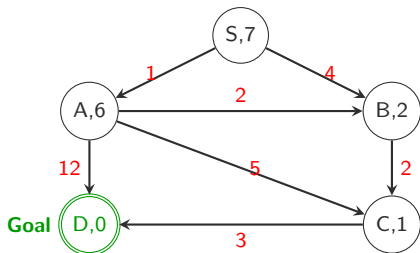
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Start Node: S

(node,g(n),h(n),f(n)), O=OPEN, C=CLOSED

St. Outcome

1. $C=\{\}, O=\{(S,0,7,7)\}$
2. $C=\{S\}, O=\{(A,1,6,7),(B,4,2,6)\}$
3. $C=\{S,B\}, O=\{(A,1,6,7),(C,6,1,7)\}$
4. $C=\{S,A\}, O=\{(B,3,2,5),(C,6,1,7),(D,13,0,13)\}$
5. $C=\{S,A,B\}, O=\{(C,5,1,6),(D,13,0,13)\}$
6. $C=\{S,A,B,C\}, O=\{(D,8,0,8)\}$
7. $C=\{S,A,B,C,D\}$

Goal D found

Terminate with cost = 8

A* Search-Example 1



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

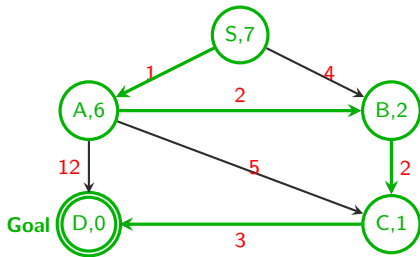
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Start Node: S

(node,g(n),h(n),f(n)), O–OPEN, C–CLOSED

St. Outcome

1. $C=\{\}, O=\{(S,0,7,7)\}$
2. $C=\{S\}, O=\{(A,1,6,7),(B,4,2,6)\}$
3. $C=\{S,B\}, O=\{(A,1,6,7),(C,6,1,7)\}$
4. $C=\{S,A\}, O=\{(B,3,2,5),(C,6,1,7),(D,13,0,13)\}$
5. $C=\{S,A,B\}, O=\{(C,5,1,6),(D,13,0,13)\}$
6. $C=\{S,A,B,C\}, O=\{(D,8,0,8)\}$
7. $C=\{S,A,B,C,D\}$

Goal D found

Terminate with cost = 8

Backtracking: $D \leftarrow C \leftarrow B \leftarrow A \leftarrow S$

Optimal Path: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$

A* Search – Optimal Path (Backtracking)



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

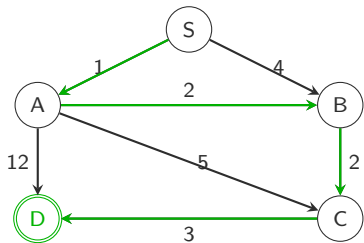
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Optimal Path Highlighted

Backtracking from Goal Node

When the goal node is reached, we trace back using the **parent pointers** stored during node expansion.

Backtracking Steps

$$D \leftarrow C \leftarrow B \leftarrow A \leftarrow S$$

Reverse the order to obtain the path.

Optimal Path

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$$

Total Cost

$$1 + 2 + 2 + 3 = 8$$

A* Search – Example 2



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

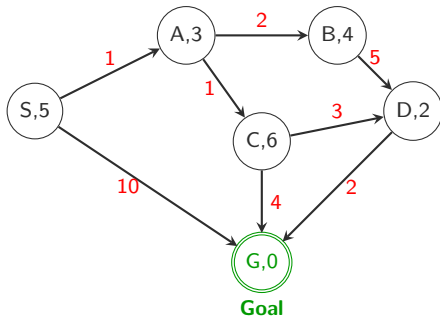
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Start Node: S

A* Search Problem

Apply the **A* Search Algorithm** to find the optimal path cost from the **Start node S** to the **Goal node G**.

Use the evaluation function:

$$f(n) = g(n) + h(n)$$

- ◇ $g(n)$: cost from start node
- ◇ $h(n)$: heuristic estimate
- ◇ $f(n)$: evaluation value

Tasks

- ◇ Show OPEN and CLOSED lists
- ◇ Compute $g(n)$, $h(n)$, $f(n)$
- ◇ find total cost

A* Search -Example 2



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

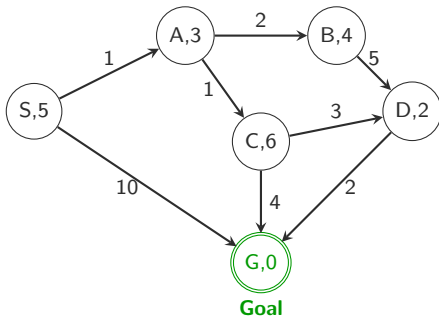
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Start Node: S

(node,g(n),h(n),f(n)), O–OPEN, C–CLOSED

St. Outcome

1. $C=\{\}, O=\{(S,0,5,5)\}$

2. $C=\{S\}, O=\{(A,1,3,4),(G,10,0,10)\}$

3. $C=\{S,A\}, O=\{(B,3,4,7),(C,2,6,8),(G,10,0,10)\}$

4. $C=\{S,A,B\}, O=\{(C,2,6,8),(D,8,2,10),(G,10,0,10)\}$

5. $C=\{S,A,B,C\}, O=\{(G,6,0,6),(D,5,2,7)\}$

6. $C=\{S,A,B,C,G\}, O=\{(D,5,2,7)\}$

Goal G found

Terminate with cost = 6



A* Search -Example 2

Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

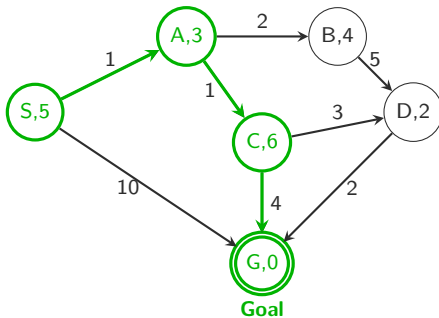
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



(node,g(n),h(n),f(n)), O–OPEN, C–CLOSED

St. Outcome

1. C={}, O={(S,0,5,5)}

2. C={S}, O={(A,1,3,4),(G,10,0,10)}

3. C={S,A}, O={(B,3,4,7),(C,2,6,8),(G,10,0,10)}

4. C={S,A,B}, O={(C,2,6,8),(D,8,2,10),(G,10,0,10)}

5. C={S,A,B,C}, O={(G,6,0,6),(D,5,2,7)}

6. C={S,A,B,C,G}, O={(D,5,2,7)}

Goal G found

Terminate with cost = 6

Backtracking: $G \leftarrow C \leftarrow A \leftarrow S$

Optimal Path: $S \rightarrow A \rightarrow C \rightarrow G$



Heuristic Properties in A* Search

Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

Heuristic Function

A heuristic $h(n)$ estimates the cost from node n to the goal. A* evaluates nodes using:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the path cost from start to n .

1. Admissible Heuristic

A heuristic is admissible if it never overestimates the true cost.

$$h(n) \leq h^*(n)$$

where $h^*(n)$ is the true optimal cost from n to the goal.

Property

- ◇ Guarantees A* finds an **optimal solution**.
- ◇ Always optimistic estimate.

2. Consistent Heuristic

A heuristic is consistent if for every node n and successor m :

$$h(n) \leq c(n, m) + h(m)$$

where $c(n, m)$ is the step cost.

Properties

- ◇ $f(n)$ values never decrease along a path.
- ◇ Node expansion happens only once.
- ◇ Every consistent heuristic is **admissible**.



Time and Space Complexity of A* Search

Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

Time Complexity

- ◇ In the worst case A* expands all nodes with

$$f(n) < C^*$$

where C^* is the cost of the optimal solution.

- ◇ If $h(n) = 0$, A* behaves like **Uniform Cost Search**.
- ◇ Worst-case time complexity:

$$O(b^m)$$

Space Complexity

- ◇ A* stores nodes in the **OPEN** and **CLOSED** lists.
- ◇ Memory requirement grows exponentially.
- ◇ Worst-case space complexity:

$$O(b^m)$$

Where

- ◇ b = branching factor
- ◇ m = maximum depth of tree
- ◇ C^* = optimal cost

A* Search – Example 3



Informed Search

Best First Search

A* Search

8-Puzzle Problem –
Introduction

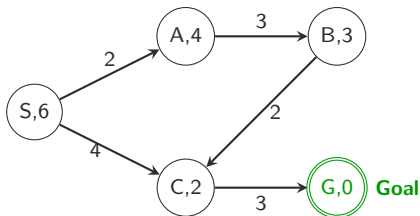
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Start Node: S

Apply A* Search

Use the evaluation function:

$$f(n) = g(n) + h(n)$$

- ◇ Show OPEN and CLOSED lists.
- ◇ Compute $g(n)$, $h(n)$ and $f(n)$ values.
- ◇ Determine the optimal path from S to G.
- ◇ Verify whether the heuristic is admissible.

A* Search – Example 3



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

(node,g(n),h(n),f(n)), O–OPEN, C–CLOSED

St. Outcome

1. $C=\{\}$, $O=\{(S,0,6,6)\}$

2. $C=\{S\}$, $O=\{(A,2,4,6),(C,4,2,6)\}$

3. $C=\{S,A\}$, $O=\{(C,4,2,6),(B,5,3,8)\}$

4. $C=\{S,A,C\}$, $O=\{(G,7,0,7),(B,5,3,8)\}$

5. $C=\{S,A,C,G\}$

Goal G found

Terminate with cost = 7

Optimal Path:

$S \rightarrow C \rightarrow G$



Admissibility Verification

Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

Admissible Heuristic

A heuristic is admissible if it never overestimates the true cost to the goal.

$$h(n) \leq h^*(n)$$

where $h^*(n)$ is the true optimal cost from node n to the goal.

True Remaining Costs

$$\diamond \quad h^*(C) = 3$$

$$(C \rightarrow G)$$

$$\diamond \quad h^*(B) = 5$$

$$(B \rightarrow C \rightarrow G)$$

$$\diamond \quad h^*(A) = 8$$

$$(A \rightarrow B \rightarrow C \rightarrow G)$$

$$\diamond \quad h^*(S) = 7$$

$$(S \rightarrow C \rightarrow G)$$

Comparison

Node	$h(n)$	$h^*(n)$
S	6	7
A	4	8
B	3	5
C	2	3

Conclusion: Since $h(n) \leq h^*(n)$ for all nodes, the heuristic is **admissible**.

A* - Example 4



Informed Search

Best First Search

A* Search

8-Puzzle Problem – Introduction

Hill Climbing Search

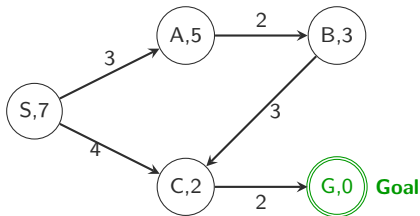
Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

Consider the following graph.
Heuristic values are given inside each node.



Start Node: S

Task

Verify whether the given heuristic is **consistent**.

A heuristic is consistent if:

$$h(n) \leq c(n, m) + h(m)$$

for every edge (n, m) .

Check the consistency condition for all edges and determine whether the heuristic satisfies the consistency property.



Consistency Condition

A heuristic is consistent if for every edge (n, m) :

$$h(n) \leq c(n, m) + h(m)$$

Edge Verification

Edge	$h(n)$	$c(n, m) + h(m)$	Result
$S \rightarrow A$	7	$3 + 5 = 8$	True
$S \rightarrow C$	7	$4 + 2 = 6$	False
$A \rightarrow B$	5	$2 + 3 = 5$	True
$B \rightarrow C$	3	$3 + 2 = 5$	True
$C \rightarrow G$	2	$2 + 0 = 2$	True

Conclusion

The condition fails for edge $S \rightarrow C$ since $7 > 6$. Therefore, the heuristic is **not consistent**.



8-Puzzle Problem – Introduction

Informed Search

Best First Search

A* Search

8-Puzzle Problem –
Introduction

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

What is the 8-Puzzle Problem?

The **8-Puzzle** is a classical problem in **Artificial Intelligence** used to study **state space search** and **heuristic search algorithms**.

Puzzle Description

- ◇ It consists of a **3×3 grid** containing **8 numbered tiles** and **one blank space**.
- ◇ The blank space allows tiles to **slide horizontally or vertically**.
- ◇ Only **one tile can move at a time**.

Objective

- ◇ Transform the **initial configuration** of tiles into a **goal configuration**.
- ◇ Achieve the goal using the **minimum number of moves**.

Solution Approach

- ◇ The problem can be solved using **search algorithms**.
- ◇ In this example, we will solve it using the **A* Search Algorithm**.

8-Puzzle Problem – Example



Informed Search

Best First Search

A* Search

8-Puzzle Problem –
Introduction

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Given an initial state of an 8–puzzle problem and final state to be reached

2	8	3
1	6	4
7		5

Initial State

1	2	3
8		4
7	6	5

Final State

- ◇ Find the most cost–effective path to reach the final state from initial state using **A*** Algorithm.

$$f(n) = g(n) + h(n)$$

- ◇ Consider $g(n) =$ **Depth of node**
- ◇ $h(n) =$ **Number of misplaced tiles**

8-Puzzle Problem – step 1



Informed Search

Best First Search

A* Search

8-Puzzle Problem –
Introduction

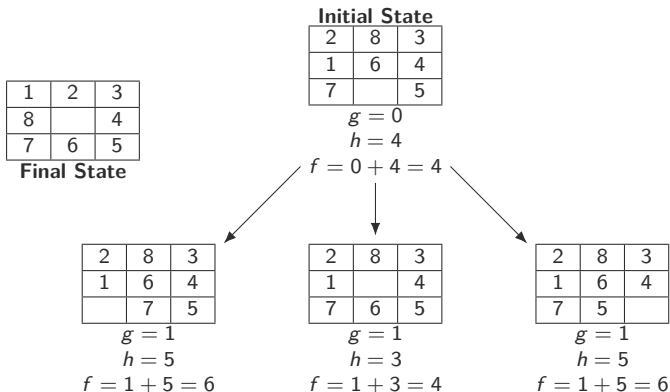
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



8-Puzzle Problem – step 2



Informed Search

Best First Search

A* Search

8-Puzzle Problem –
Introduction

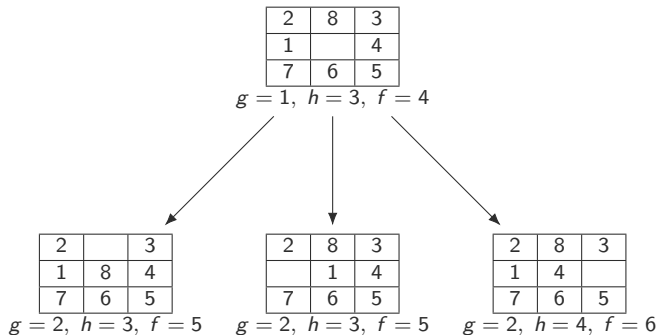
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



8-Puzzle Problem – step 3



Informed Search

Best First Search

A* Search

8-Puzzle Problem –
Introduction

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

1	2	3
8		4
7	6	5

Final State

2	8	3
	1	4
7	6	5

$$g = 2$$

$$h = 3$$

$$f = 2 + 3 = 5$$

2		3
1	8	4
7	6	5

$$g = 2$$

$$h = 3$$

$$f = 2 + 3 = 5$$

	8	3
2	1	4
7	6	5

$$g = 3$$

$$h = 3$$

$$f = 3 + 3 = 6$$

2	8	3
7	1	4
	6	5

$$g = 3$$

$$h = 4$$

$$f = 3 + 4 = 7$$

	2	3
1	8	4
7	6	5

$$g = 3$$

$$h = 2$$

$$f = 3 + 2 = 5$$

2	3	
1	8	4
7	6	5

$$g = 3$$

$$h = 4$$

$$f = 3 + 4 = 7$$

8-Puzzle Problem – step 4



Informed Search

Best First Search

A* Search

8-Puzzle Problem –
Introduction

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

1	2	3
8		4
7	6	5

Final State

	2	3
1	8	4
7	6	5

$g = 3$

$h = 2$

$f = 3 + 2 = 5$



1	2	3
	8	4
7	6	5

$g = 4$

$h = 1$

$f = 4 + 1 = 5$

8-Puzzle Problem – step 5



Informed Search

Best First Search

A* Search

8-Puzzle Problem –
Introduction

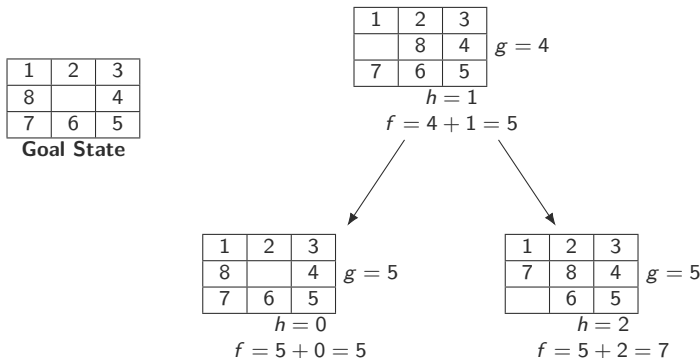
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Goal State Reached.

Since the goal node has the minimum $f(n) = 5$, the A* search terminates.

Hill Climbing Search



The Core Concept

- ◇ A **heuristic local search algorithm** used for optimization.
- ◇ Driven by a **greedy approach**: it evaluates neighbors and strictly chooses the move with the highest immediate reward.
- ◇ *Analogy*: Climbing a mountain in dense fog—you only see your immediate surroundings and always step in the steepest upward direction.

Key Characteristics

- ◇ **Memory-Efficient**: Does not maintain a search tree; stores only the **current state** and its immediate neighbors.
- ◇ **Irrevocable (No Backtracking)**: Once a move is executed, it cannot return to a previous state to try a different path.
- ◇ **Termination**: Halts the moment it reaches a peak where no neighbor offers a strictly better heuristic value.

Hill Climbing: State Transition



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

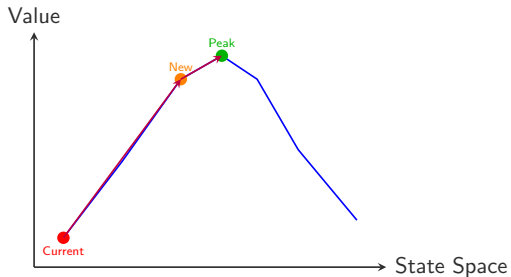
Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Decision Rule

$$f(NS) > f(CS) \Rightarrow CS = NS$$

Else: Stop



Key Idea

- ◇ Move towards better state
- ◇ Stop at peak

Hill Climbing: Flowchart



Informed Search

Best First Search

A* Search

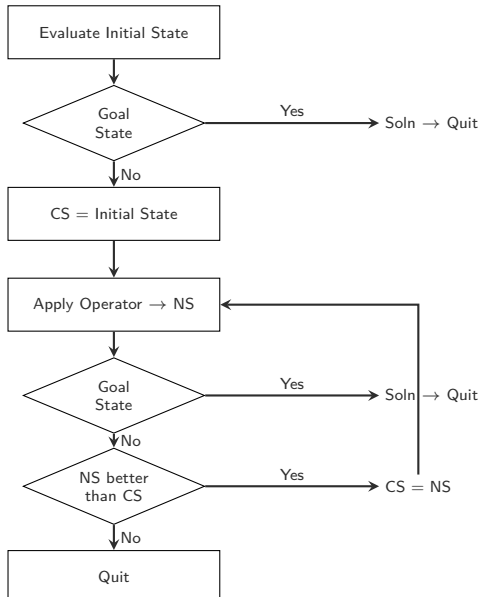
Hill Climbing
Search

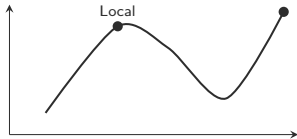
Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

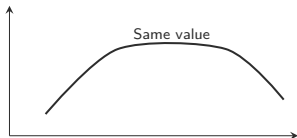
Cryptarithmic
Problems





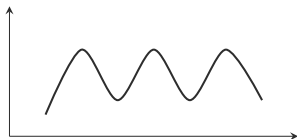
1. Local Maxima

A peak that is higher than its immediate neighbors, but not the global best. The greedy algorithm gets permanently trapped here.



2. Plateau (Flat Maximum)

A flat area where all neighbors have the exact same heuristic value. Lacking a gradient to follow, the algorithm wanders randomly or halts.



3. Ridge

A narrow, steep slope of local maxima. Because standard operators only move in one dimension at a time, it causes inefficient zig-zagging.



Question:

Apply **Hill Climbing Search** to reach the goal state from the given initial state.

Initial State

$$\begin{bmatrix} 1 & - & 7 \\ 2 & 4 & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Goal State

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & - \end{bmatrix}$$

Heuristic Used (h): **Minimization Problem**

- ◇ Number of misplaced tiles (ignoring the blank space).
- ◇ We want to minimize h until we reach the goal or get stuck at a **local minimum**.

Hill Climbing Solution (Step 1)



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Initial State ($h = 3$)

$$\begin{bmatrix} 1 & _ & 7 \\ 2 & 4 & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Goal State

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & _ \end{bmatrix}$$

Generate Neighbors (from blank at top-center)

Left ($h = 4$)

$$\begin{bmatrix} _ & 1 & 7 \\ 2 & 4 & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Right ($h = 4$)

$$\begin{bmatrix} 1 & 7 & _ \\ 2 & 4 & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Down ($h = 2$)

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & _ & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Select best ($h = 2 < 3$)

$$\Rightarrow \begin{bmatrix} 1 & 4 & 7 \\ 2 & _ & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Hill Climbing Solution (Step 2)



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Current State ($h = 2$)

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & - & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Goal State

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & - \end{bmatrix}$$

Generate Neighbors

Up ($h = 3$)

$$\begin{bmatrix} 1 & - & 7 \\ 2 & 4 & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Down ($h = 3$)

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 6 & 5 \\ 3 & - & 8 \end{bmatrix}$$

Left ($h = 3$)

$$\begin{bmatrix} 1 & 4 & 7 \\ - & 2 & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Right ($h = 1$)

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & - \\ 3 & 6 & 8 \end{bmatrix}$$

Select best ($h = 1 < 2$)

$$\Rightarrow \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & - \\ 3 & 6 & 8 \end{bmatrix}$$

Hill Climbing Solution (Step 3)



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Current State ($h = 1$)

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & - \\ 3 & 6 & 8 \end{bmatrix}$$

Goal State

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & - \end{bmatrix}$$

Generate Neighbors
Up ($h = 2$)

$$\begin{bmatrix} 1 & 4 & - \\ 2 & 5 & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Left ($h = 2$)

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & - & 5 \\ 3 & 6 & 8 \end{bmatrix}$$

Down ($h = 0$)

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & - \end{bmatrix}$$

Select best ($h = 0 < 1$)

$$\Rightarrow \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & - \end{bmatrix}$$



Final State Reached ($h = 0$)

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & - \end{bmatrix}$$

Observation

- ◇ The algorithm successfully found a path where every step strictly improved the heuristic ($3 \rightarrow 2 \rightarrow 1 \rightarrow 0$).
- ◇ The Search stops at Step 3 because $h = 0$ indicates the **Goal State**.
- ◇ This is an example of Hill Climbing efficiently reaching the global optimum without getting trapped!



Question:

Apply **Hill Climbing Search** to reach the goal state from the given initial state using a suitable heuristic.

Initial State

$$\begin{bmatrix} 1 & 2 & 4 \\ 5 & - & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Goal State

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & - \end{bmatrix}$$

Heuristic Used:

- ◇ Number of misplaced tiles (or any suitable heuristic)

Hill Climbing Solution (Step 1)



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Initial State ($h = 5$)

$$\begin{bmatrix} 1 & 2 & 4 \\ 5 & _ & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Goal State

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & _ \end{bmatrix}$$

Generate Neighbors

Left ($h = 4$)

$$\begin{bmatrix} 1 & 2 & 4 \\ _ & 5 & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Right ($h = 5$)

$$\begin{bmatrix} 1 & 2 & 4 \\ 5 & 7 & _ \\ 3 & 6 & 8 \end{bmatrix}$$

Up ($h = 5$)

$$\begin{bmatrix} 1 & _ & 4 \\ 5 & 2 & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Down ($h = 6$)

$$\begin{bmatrix} 1 & 2 & 4 \\ 5 & 6 & 7 \\ 3 & _ & 8 \end{bmatrix}$$

Select best ($h = 4 < 5$)

$$\Rightarrow \begin{bmatrix} 1 & 2 & 4 \\ _ & 5 & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Hill Climbing Solution (Step 2)



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Current State ($h = 4$)

$$\begin{bmatrix} 1 & 2 & 4 \\ - & 5 & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Goal State

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & - \end{bmatrix}$$

Generate Neighbors

Up ($h = 5$)

$$\begin{bmatrix} - & 2 & 4 \\ 1 & 5 & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Right ($h = 5$)

$$\begin{bmatrix} 1 & 2 & 4 \\ 5 & - & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Down ($h = 5$)

$$\begin{bmatrix} 1 & 2 & 4 \\ 3 & 5 & 7 \\ - & 6 & 8 \end{bmatrix}$$

No better state found ($h \geq 4$)



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Final State

$$\begin{bmatrix} 1 & 2 & 4 \\ - & 5 & 7 \\ 3 & 6 & 8 \end{bmatrix}$$

Observation

- ◇ No neighbor improves heuristic
- ◇ Algorithm stops at Step 2
- ◇ Example of **local minimum**



Branching Factor (b)

- ◇ b = number of **neighboring states** from current state

Time Complexity

- ◇ If k = number of steps:
 $\mathcal{O}(k \cdot b)$
- ◇ Depends on steps until no better neighbor exists

Space Complexity

- ◇ Stores only current state
 $\mathcal{O}(1)$

Optimality & Completeness

- ◇ **Not Optimal:** May stop at local maxima
- ◇ **Not Complete:** May fail even if solution exists

Game Tree Search: The Mini-Max Algorithm



Core Concept

- ◇ A **recursive backtracking algorithm** used for decision-making in game theory.
- ◇ Designed for **two-player, zero-sum games** where one player's gain is exactly equal to the opponent's loss.
- ◇ *Goal:* To determine the optimal sequence of moves by assuming both players play perfectly.

The Players

- ◇ **MAX:** Aims to reach a state with the **maximum** possible score.
- ◇ **MIN:** Aims to minimize MAX's advantage by choosing moves with the **minimum** value.



Exploration Mechanism

- ◇ Operates as a **Depth-First Search (DFS)** to explore the entire game tree down to the terminal nodes.
- ◇ Back-propagates values from the leaves to the root to compute the best move for the current state.

Initial Heuristic Bounds

Player	Initial Value	Logic
MAX	$-\infty$	Starting with the worst possible value to find better moves.
MIN	$+\infty$	Starting with the highest value to find lower alternatives.

Examples: Chess, Checkers, and Tic-Tac-Toe.



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

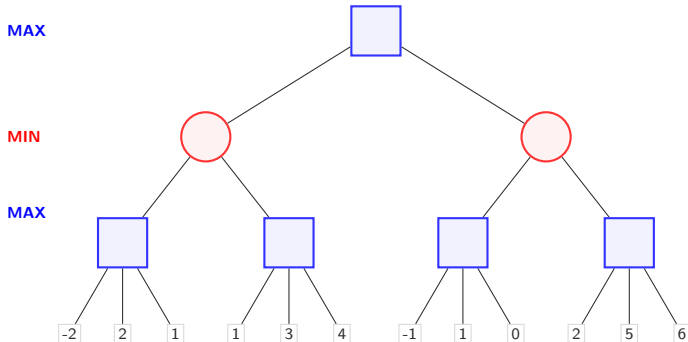
Theoretical Properties

- ◇ **Completeness:** Definitely finds a solution if the tree is finite.
- ◇ **Optimality:** Guaranteed to find the optimal strategy against an ideal opponent.
- ◇ **Time Complexity:** $O(b^m)$ — Branching factor b raised to depth m (maximum depth of tree).
- ◇ **Space Complexity:** $O(bm)$ — Linear space required for DFS stack.

Major Limitations

- ◇ **Search Space Explosion:** For games like Chess, the number of nodes makes full exploration impossible.
- ◇ **Horizon Effect:** Decisions are limited by the search depth, potentially missing critical future states.
- ◇ *Standard Optimization:* Usually implemented with **Alpha-Beta Pruning** to improve efficiency.

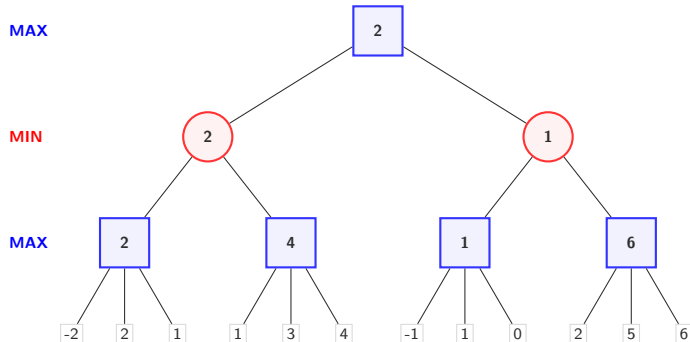
Mini-Max Algorithm: Example 1



Problem Statement:

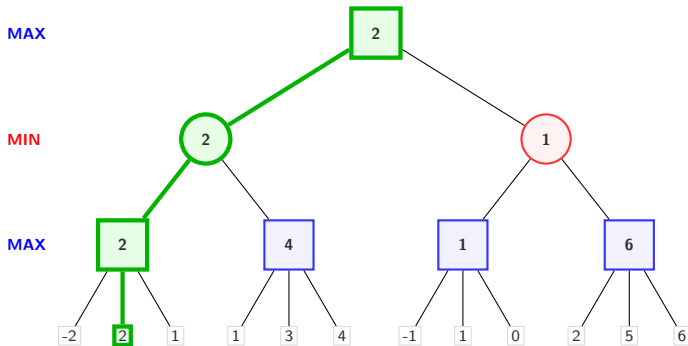
- ◇ **Goal:** Determine the optimal value at the root node using the Mini-Max decision rule.
- ◇ **Constraints:** Assume both players (MAX and MIN) play optimally.
- ◇ **Initial Search:** Values are initialized at the leaf nodes (Level 3).

Mini-Max Algorithm: Example 1



- ◇ **Optimal Selection:** The root (MAX) chooses the left path because it yields a higher guaranteed value ($2 > 1$).
- ◇ **Heuristic Propagation:** Values are determined at the leaves and moved upward; MIN players choose the minimum child, while MAX players choose the maximum.

Mini-Max Algorithm: optimal path



- ◇ **Optimal Path:** Highlighted in green, leading to the best guaranteed outcome.
- ◇ **Final Decision:** The root (MAX) chooses the path that results in a final value of 2.

Mini-Max Algorithm: Example 2



Informed Search

Best First Search

A* Search

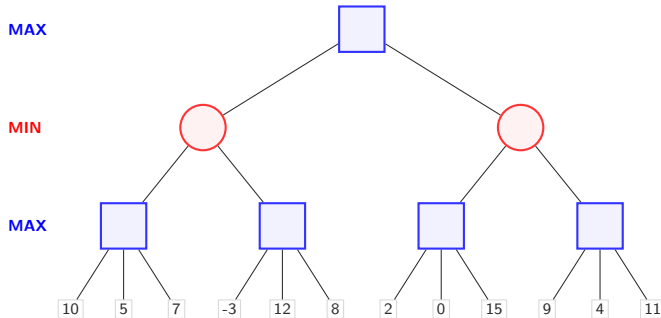
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

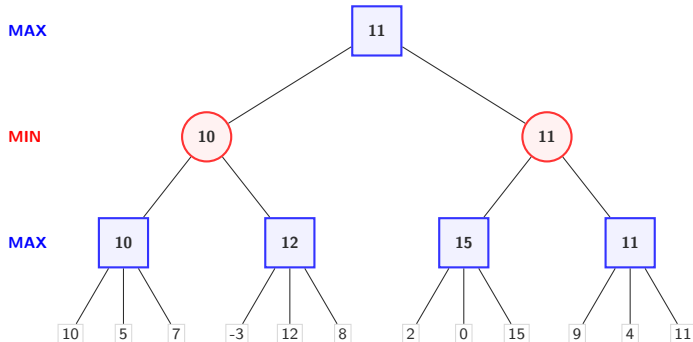
Cryptarithmic
Problems



Exercise:

- ◇ Apply the Mini-Max decision rule to find the optimal root value.
- ◇ Assume optimal play from both the MAX and MIN players.

Mini-Max Algorithm: Example 2



- ◇ **Optimal Selection:** The root (MAX) evaluates the outcomes from the left sub-tree (10) and the right sub-tree (11), ultimately choosing the right path.
- ◇ **Heuristic Propagation:** Values are determined at the leaf nodes and moved upward; MIN players choose the minimum child, while MAX players choose the maximum.

Mini-Max: Optimal Path



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

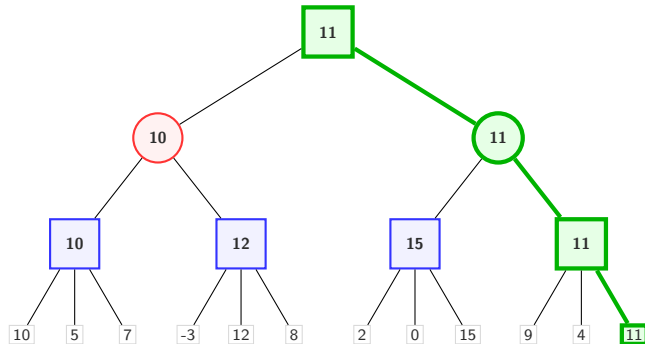
Constraint
Satisfaction
Problem

Cryptarithmic
Problems

MAX

MIN

MAX



- ◇ **Optimal Decision:** The green path highlights the sequence where MAX achieves the highest guaranteed value of **11**.
- ◇ **Logic Trace:** Root (MAX) chooses Right ($11 > 10$), MIN chooses Right ($11 < 15$), and MAX chooses Rightmost leaf ($11 > 9, 4$).

Alpha-Beta Pruning



Overview

- ◇ An optimization technique for the **Minimax algorithm**.
- ◇ Goal: To reduce the number of nodes evaluated in the search tree.
- ◇ Does not change the final result; it simply finds the optimal move faster.

The Key Parameters

- ◇ α (**Alpha**): The best (highest) value found so far for the **MAX** player.
- ◇ β (**Beta**): The best (lowest) value found so far for the **MIN** player.

Pruning Logic

- ◇ As the tree is traversed, we update $[\alpha, \beta]$.
- ◇ If at any point $\alpha \geq \beta$, the current player knows the opponent will never allow this path to be reached.
- ◇ **Action:** The remaining branches are **pruned** (skipped).

Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

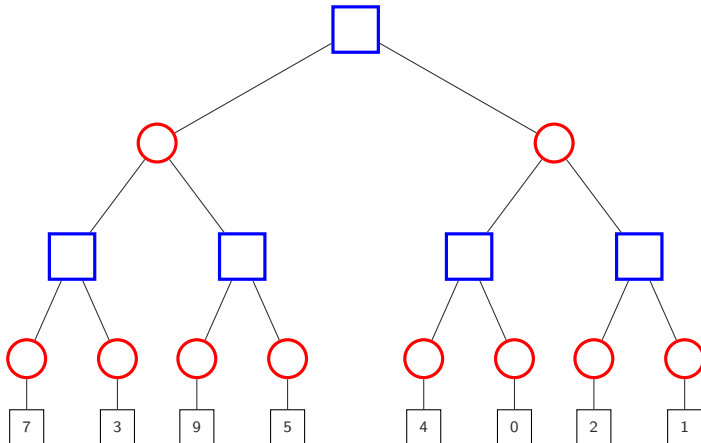
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Question: Apply the Alpha-Beta Pruning algorithm to the game tree above. Show the final minimax value at the root node, the values computed at each node, and clearly indicate all pruned branches.

Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

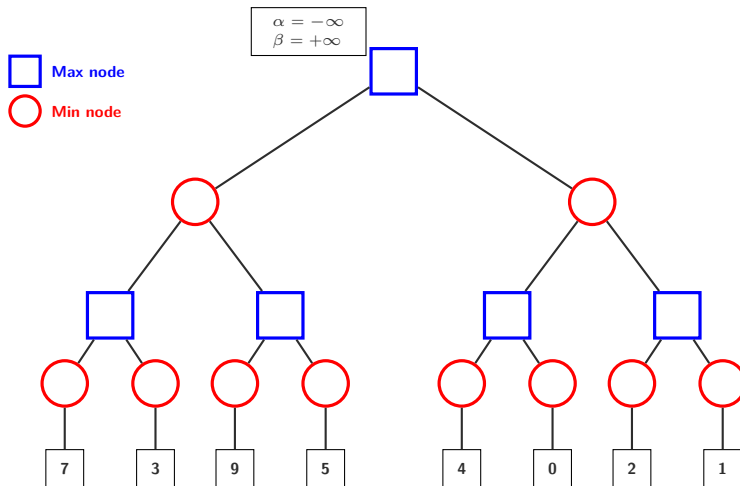
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

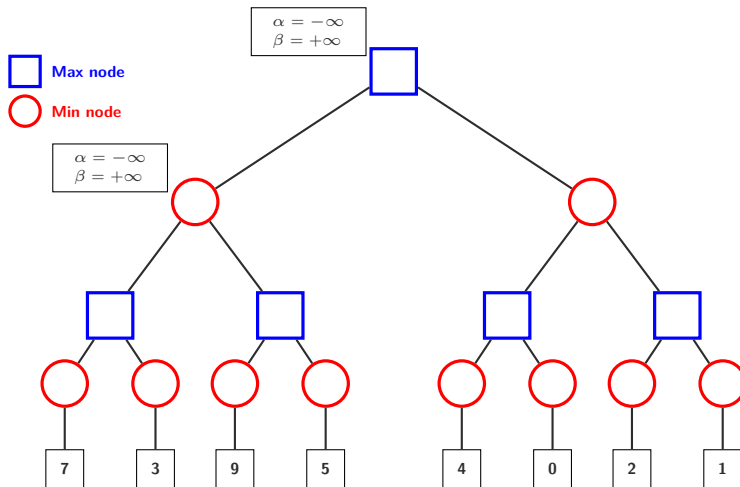
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

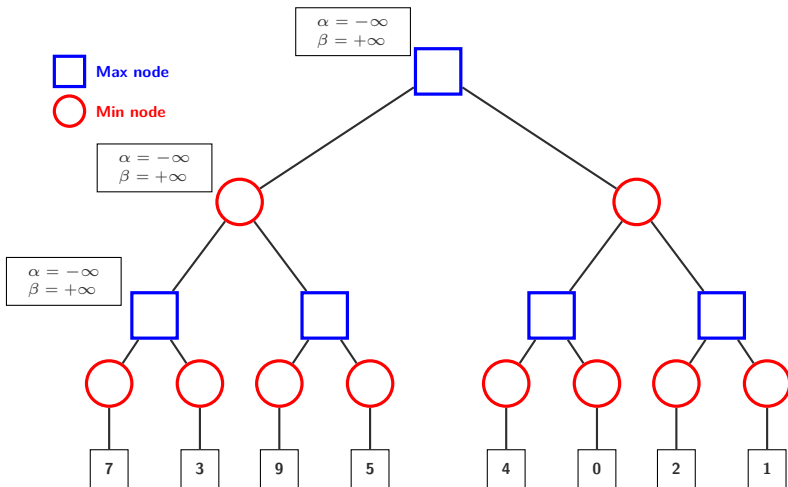
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

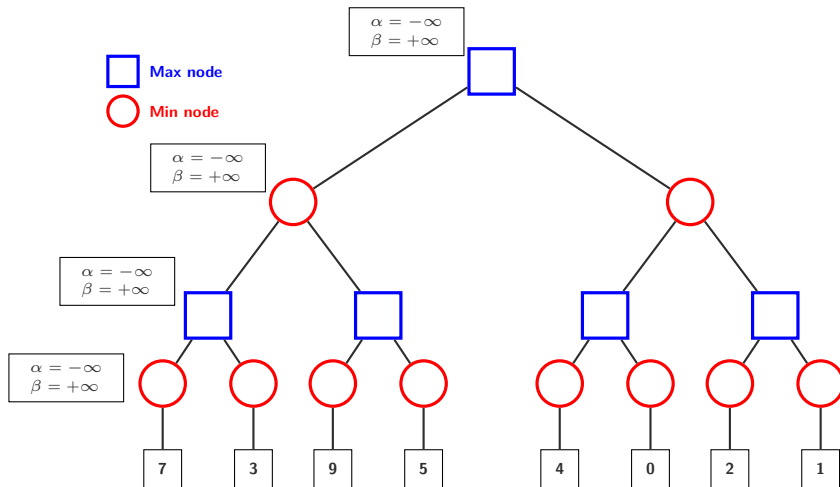
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

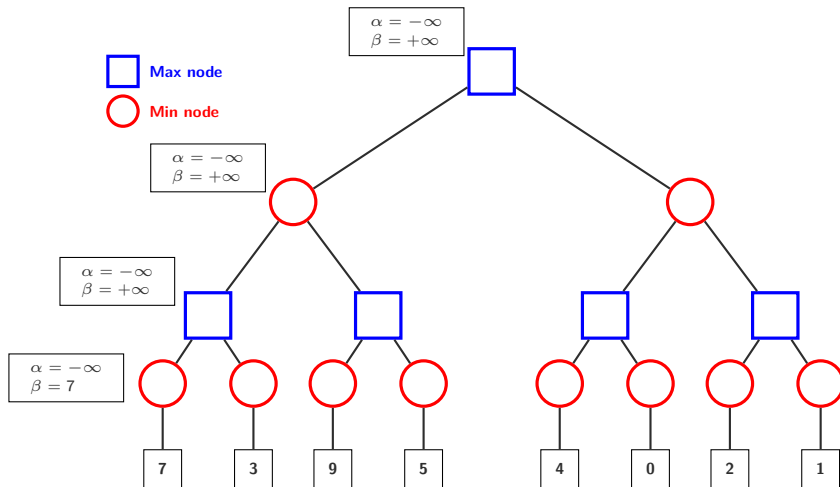
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

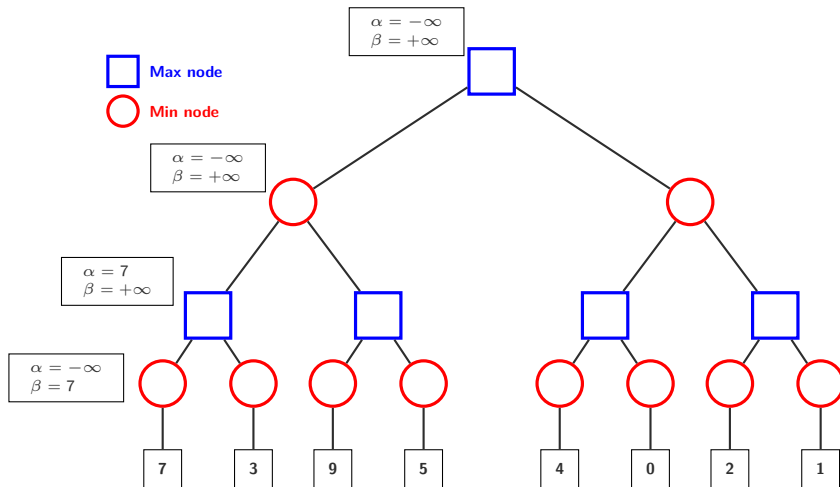
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

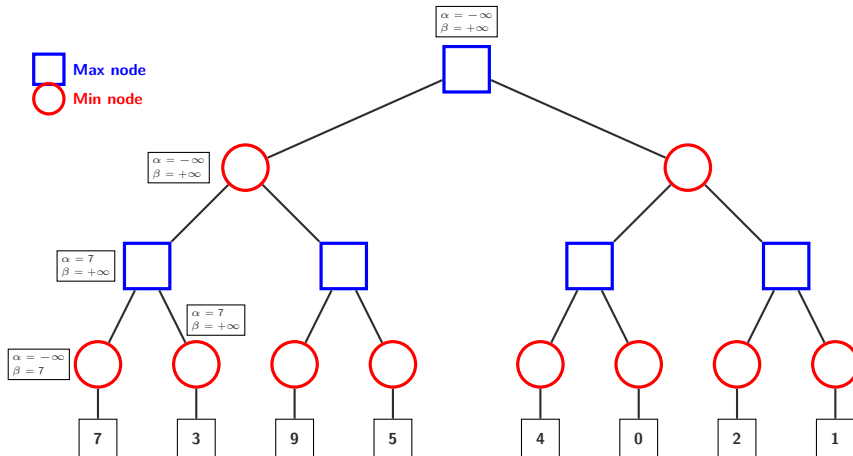
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

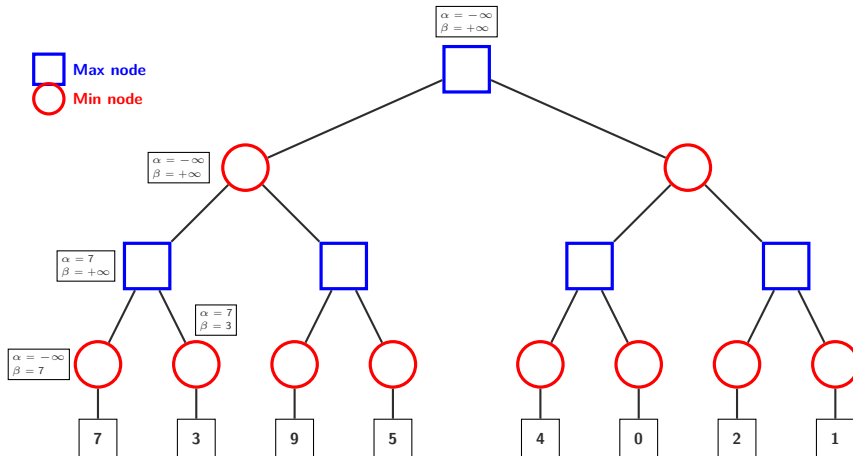
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems











Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

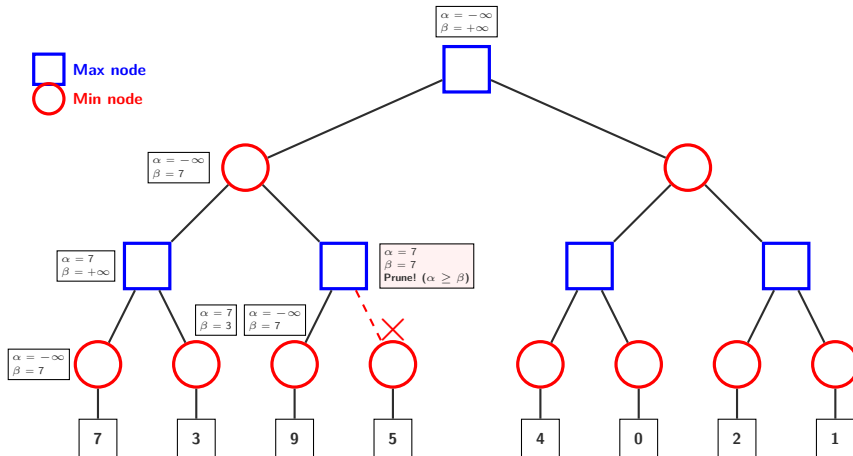
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems





Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

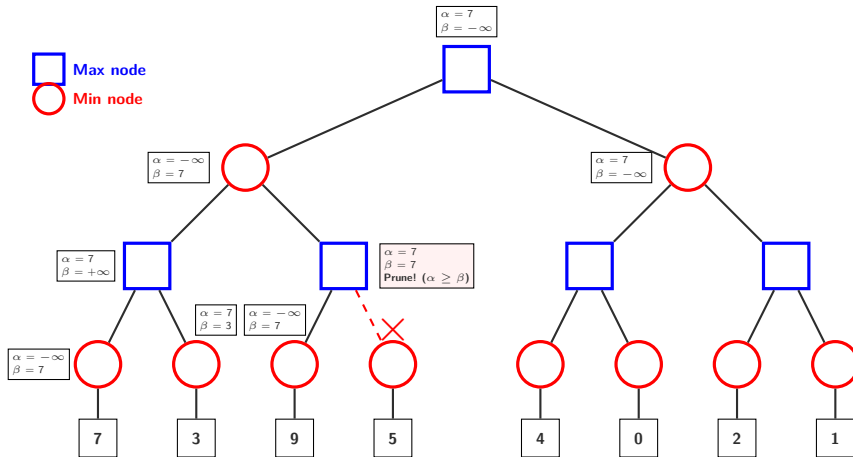
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

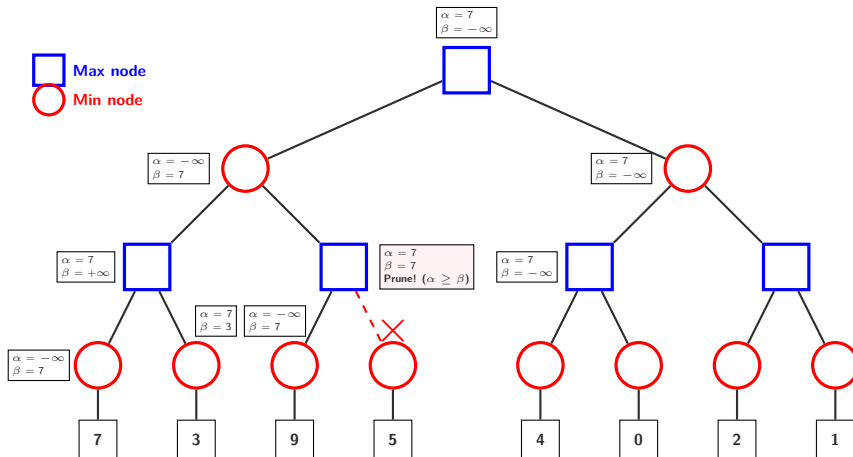
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

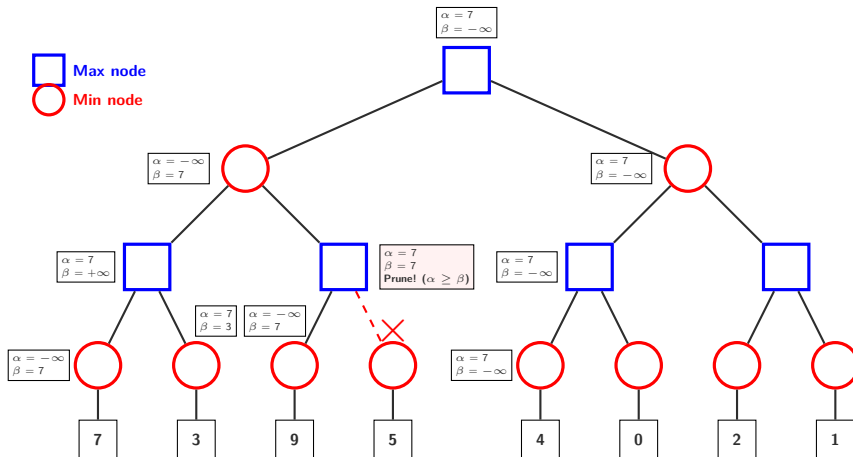
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

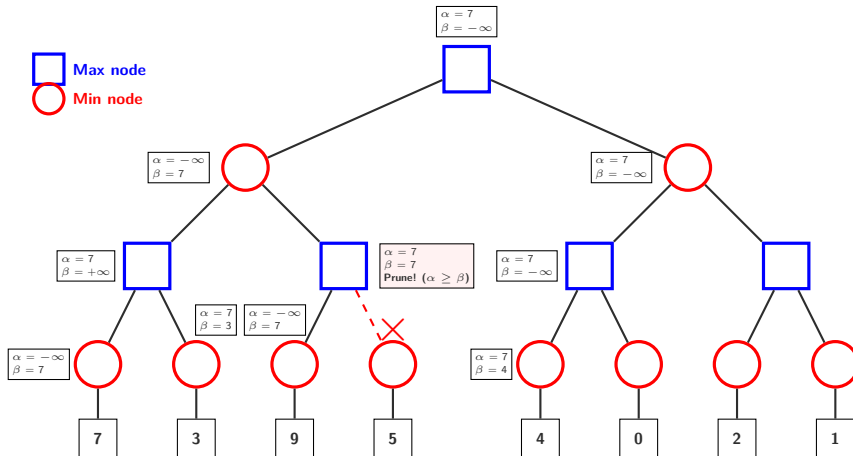
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

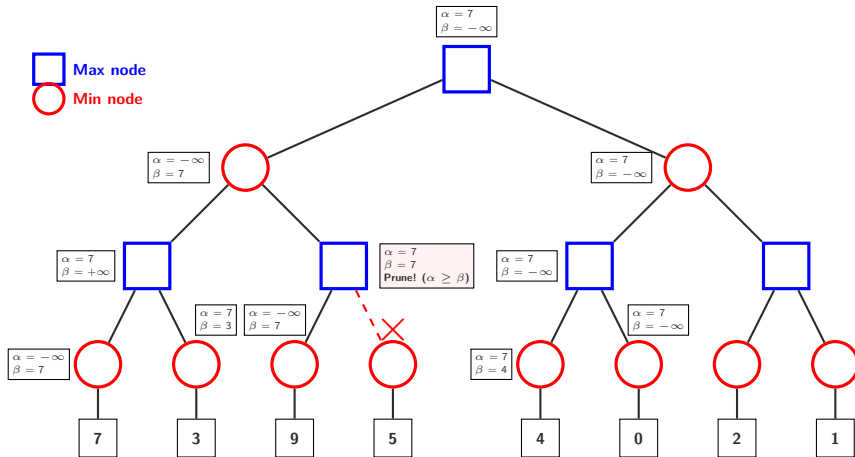
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

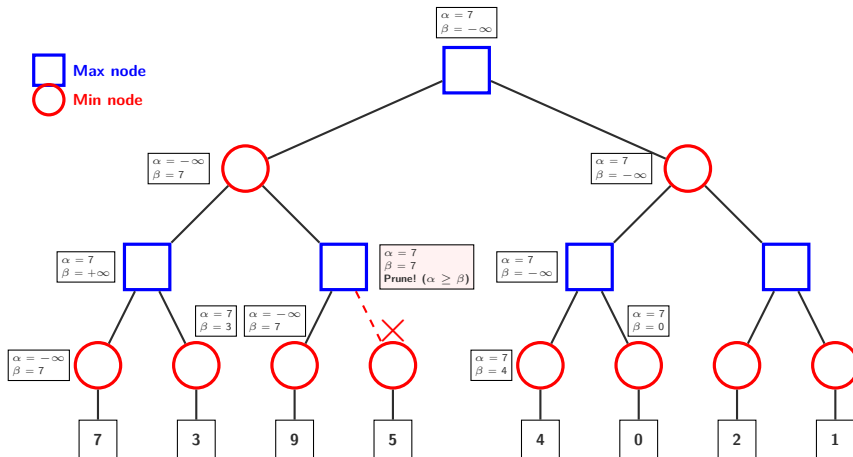
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems





Alpha-Beta Pruning



Informed Search

Best First Search

A* Search

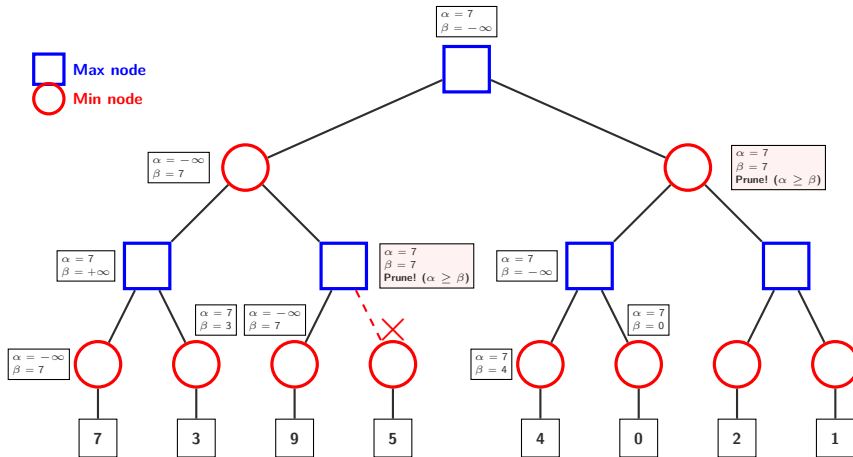
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems





Alpha-Beta Pruning: Practice Question



Informed Search

Best First Search

A* Search

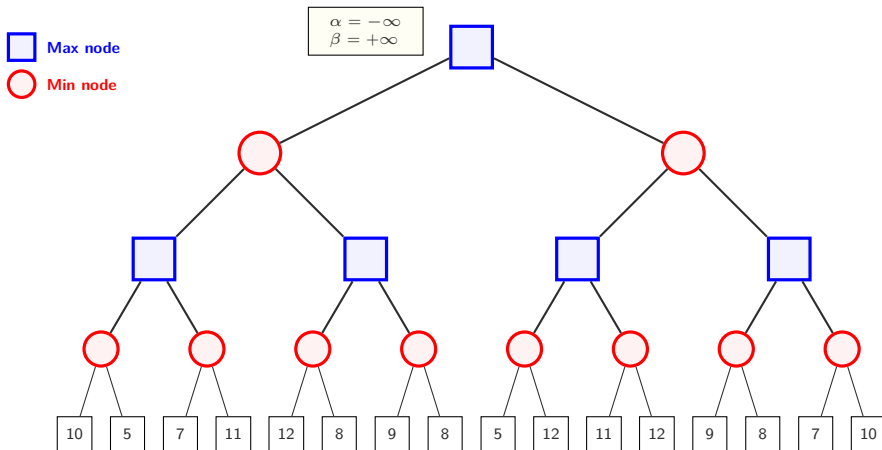
Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems



Alpha-Beta Pruning: Practice Question (sol)



Informed Search

Best First Search

A* Search

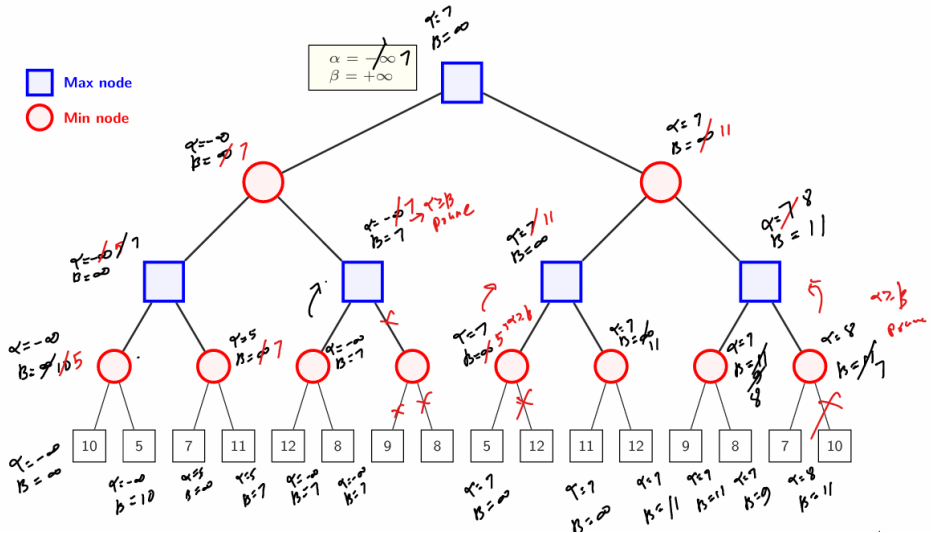
Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems



Constraint Satisfaction Problem



Constraint Satisfaction

Informed Search

Best First Search

A* Search

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

Constraint Satisfaction:–

- ◇ It is a **Search procedure** that operates in a **Space of Constraint Sets**.
- ◇ **Constraint Satisfaction Problems** in AI have goal of discovering some problem state that satisfies a **given set of constraints**.

PROCESS:

- Constraints are discovered and propagated throughout the system.**
 - If still there is no solⁿ, search begins.
- ◇ A **Guess** is made about something and added as new Constraint.

CSP

- $\underline{\mathbf{V}} \{V_1, V_2, \dots, V_n\}$ Variable set
- $\underline{\mathbf{D}} \{D_1, D_2, \dots, D_n\}$
one domain for each Variable
- $\underline{\mathbf{C}}$ (Constraints)
Specify allowable combination of values.

$C_i = (\text{Scope}, \text{Relation})$

- ◇ Scope: Set of Variables that participate in Constraint.
- ◇ Relation: Defines Values that a Variable can take.

Example: Graph Node Coloring



Informed Search

Best First Search

A* Search

Hill Climbing
Search

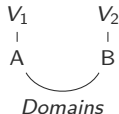
Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Variable Domains



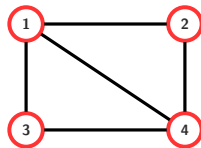
(1) Values of V_1 and V_2 Can't be Same

◇ $C_1 = \{(V_1, V_2), V_1 \neq V_2\}$

Solving Methodology

- ◇ → **Intelligent Backtracking** method is used to solve CSP.
- ◇ ↪ only backtrack where Conflict occurs.

NODE COLORING GRAPH



CSP Problem Formulation

- ◇ **Variables (V):** Nodes $\{1, 2, 3, 4\}$.
- ◇ **Domain (D):** Colors $\{R, G, B\}$ per node.
- ◇ **Constraints (C):** Adjacent nodes should not be same color.

Solution Trace: Node Coloring with Backtracking



Informed Search

Best First Search

A* Search

Hill Climbing Search

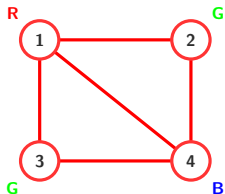
Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

NODE COLORING



Assignment Logic Table

	1	2	3	4	
Initial	R,G,B	R,G,B	R,G,B	R,G,B	
1 = R	R	G,B	G,B	G,B	
2 = G	R	G	G,B	B	
3 = B	R	G	B	—	ERROR
3 = G	R	G	G	B	Valid

- ◇ **V:** {1, 2, 3, 4}
- ◇ **D:** {R, G, B}
- ◇ **C:** Adjacent nodes $\neq$ same color.

- ◇ **Intelligent Backtracking:**
- ◇ When **3 = B** failed for Node 4, we reverted to change Node 3's value.
- ◇ **Action:** Assignment changed to **3 = G**.

Problem: Examination Scheduling



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

The Challenge

- Subjects: $\{C_1, C_2, C_3, C_4, C_5, C_6\}$.
- Constraint: A student cannot take more than one exam per day.

Your Tasks:

1. Define **CSP Formulation**.
2. Construct **Constraint Graph**.
3. Find minimum days required.

Student Enrollment Data

Student	Subjects (Exams)
S_1	C_1, C_2, C_3
S_2	C_2, C_3, C_4
S_3	C_3, C_4
S_4	C_3, C_4, C_5
S_5	C_1, C_5, C_6

Hint: Each student represents binary constraints between subjects.

Question: Is it possible to conduct all exams in **3 days** assuming one exam per day?

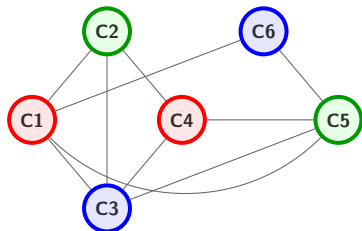
1) CSP Formulation (V, D, C)

- ◇ **Variables (V):**
 $\{C_1, C_2, C_3, C_4, C_5, C_6\}$
- ◇ **Domain (D):** $\{Day_1, Day_2, Day_3\}$
- ◇ **Constraints (C):** $C_i \neq C_j$ if they share a student.

2) Conflict Pairs (Edges)

- ◇ **S1:** $(C_1, C_2), (C_1, C_3), (C_2, C_3)$
- ◇ **S2:** $(C_2, C_4), (C_3, C_4)$
- ◇ **S4:** $(C_3, C_5), (C_4, C_5)$
- ◇ **S5:** $(C_1, C_5), (C_1, C_6), (C_5, C_6)$

Validated Constraint Graph (3-Colored)



Result: A valid 3-coloring exists, proving the minimum days required is **3**.

Cryptarithmic Problems



Overview

- ◇ A specific type of **Constraint Satisfaction Problem (CSP)**.
- ◇ Mathematical puzzles where digits are replaced by **alphabets/symbols**.
- ◇ The goal is to decode the digits while satisfying arithmetic rules.

Core Constraints

- ◇ **Unique Assignment:** No two different letters can have the same numerical value.
- ◇ **Arithmetic Sum:** The sum of the digits must be exactly as shown in the problem.
- ◇ **Digit Range:** Each letter must be assigned a single digit from the range **0–9**.
- ◇ **Carry Forward:** There should be only one carry forward per column (usually 0 or 1).

Problem Representation

- ◇ A typical problem is represented as a vertical addition:

$$\begin{array}{r} \\ + \\ \hline 0 \end{array}$$

Hint: The leftmost carry (O) will be 1.



Solution Trace: $TO + GO = OUT$

Informed Search

Best First Search

A* Search

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

Step 1: The Units Column

- Equation: $O + O = T$
- Substitute $O = 1$: $1 + 1 = 2$.
- Assignment:** Letter **T** = 2.

Step 2: The Tens Column

- Equation: $T + G = U \Rightarrow 2 + G = U$.
- This means $2 + G = 10 + U$.
- We must choose a digit for **G** (Range 0-9):
- If **G** = 8: $2 + 8 = 10 \rightarrow \mathbf{U} = 0$.
- (Note: If $G = 9$, $U = 1$, which conflicts with $O = 1$).
- Assignment:** **G** = 8, **U** = 0.

Final Decoded Addition

$$\begin{array}{r}
 21 \quad (TO) \\
 + 81 \quad (GO) \\
 \hline
 102 \quad (OUT)
 \end{array}$$

Letter to Digit Map:

$T \rightarrow 2$
 $O \rightarrow 1$
 $G \rightarrow 8$
 $U \rightarrow 0$

All constraints satisfied: Each letter has a unique digit and the sum is correct.

Problem: SEND + MORE = MONEY



Informed Search

Best First Search

A* Search

Hill Climbing
Search

Game Tree
Search: The
Mini-Max
Algorithm

Alpha-Beta
Pruning

Constraint
Satisfaction
Problem

Cryptarithmic
Problems

Problem Statement

- ◇ This is a classic base-10 cryptarithmic puzzle.
- ◇ Each letter represents a unique digit from $\{0, 1, \dots, 9\}$.
- ◇ No two letters can share the same digit.
- ◇ The leading letters (**S** and **M**) cannot be zero.

Solve the Addition:

$$\begin{array}{rcccc} & \mathbf{S} & \mathbf{E} & \mathbf{N} & \mathbf{D} \\ + & & & & \\ \hline \mathbf{M} & \mathbf{O} & \mathbf{N} & \mathbf{E} & \mathbf{Y} \end{array}$$

Your Tasks:

1. Define the Variables (V) and Domain (D).
2. Decode the numerical values for all 8 letters.

Hint: Start with the leftmost carry. the leftmost carry (M) will be 1.



Solution Trace: SEND + MORE = MONEY

Informed Search

Best First Search

A* Search

Hill Climbing Search

Game Tree Search: The Mini-Max Algorithm

Alpha-Beta Pruning

Constraint Satisfaction Problem

Cryptarithmic Problems

Step 1: Solve for S and O

- Column 4: $S + M + (\text{carry}) = 10 + O$.
- Since $M = 1$, $S + 1 + (\text{carry}) = 10 + O$.
- If $S = 9$ and $\text{carry} = 0$, $9 + 1 + 0 = 10$. then $O = 0$.

Step 2: Solve for E, N, R

- From Col 3: $E + 0 + (\text{carry}) = N$.
- From Col 2: $N + R + (\text{carry}) = 10 + E$.
- Testing values (Constraint: all digits unique):
- $E = 5$, $N = 6$, $R = 8$.**

Step 3: Units Column ($D + E = Y$)

- $7 + 5 = 12 \rightarrow Y = 2$ (Carry 1 to $N + R$).
- $D = 7$, $Y = 2$.**

Final Decoded Addition

$$\begin{array}{r}
 9 5 6 7 \quad (S \ E \ N \ D) \\
 + 1 0 8 5 \quad (M \ O \ R \ E) \\
 \hline
 1 0 6 5 2 \quad (M \ O \ N \ E \ Y)
 \end{array}$$

Letter to Digit Map:

S:9, E:5, N:6, D:7
M:1, O:0, R:8, Y:2

All digits are unique $\{0, 1, 2, 5, 6, 7, 8, 9\}$ and the arithmetic is valid.

Thank you for your attention!